AI-Copilot

可扩展 Agent 工作台平台

产品需求文档 PRD V3.0

完整融合版 · 工程级落地规格

字段	内容
文档版本	V3.0（V1.0 + V2.0 深度融合，工程级完整规格）
平台定位	可扩展 Agent 工作台平台，首期搭载结案报告 Skill
部署形态	Web 应用（React 18），内网服务器，浏览器直接访问，零客户端安装
LLM 策略	后端可插拔（Claude / OpenAI / Ollama / 自定义），混合部署
工程方法	两级 Agent + Skill 注册协议 + Harness CI/CD + 经验层
AI 辅助开发	Claude Code CLI 全程辅助，含分阶段提示词指南
适用读者	PM · 架构师 · 后端开发 · 前端开发 · 运维 · QA · 业务方

一、文档导读

章节	核心内容	主要读者
一	文档导读与版本说明	全体
二	平台背景、痛点、核心目标与验收标准	PM · 业务方
三	用户角色与权限矩阵	PM · 后端
四	系统架构：三层结构 / 两级 Agent / HIL 机制 / 状态机	架构师 · 后端
五	平台层详细规格：LLM Adapter / Tool 调度 / 安全基线 / 可观测性	后端
六	Skill 注册协议与扩展规范	所有 Skill 开发者
七	首期 Skill A：活动结案报告（完整业务规格 + 所有 Tool）	业务方 · 后端 · 前端
八	经验层：反馈采集 / 模式挖掘 / Prompt 调优 / 工具发现 / 分支管理	PM · Admin
九	Harness 工程结构与 CI/CD 方案	运维 · 后端
十	数据库完整设计（平台表 + Skill A 表）	后端
十一	部署架构与环境配置	运维
十二	非功能性需求、安全规范、UAT 验收场景	QA · PM
十三	AI 辅助开发实操指南（Claude Code CLI）	全体开发
十四	未来 Skill 扩展路线图	PM · 架构师

二、平台背景与目标

2.1 业务痛点

活动执行团队在项目结案时面临以下高频痛点：

痛点	具体表现	量化影响
照片处理耗时	单项目需 2～6 小时在数百张乱序现场照片中肉眼寻找物料、截图、排版到 PPT	人均每月损耗 40～120 小时
多源资产混乱	报价单 Excel、设计效果图（PPT/文件夹）、现场照片（本地/云相册混存）三类资产命名乱序，无法对应	准备阶段额外耗时 30～60 分钟/项目
极易出错漏配	纯人工排版导致漏配图、物料数量对不上，引发财务扣款或返工	每次返工 2～4 小时，月均 2～4 次
凭据核对繁琐	支付截图（微信/支付宝）和发票 PDF 需手动比对金额与日期，容易遗漏	每项目凭据核对 1～2 小时
月末并发崩溃	集中结案期并发压力大，业务员大规模加班	月末 3 天加班率超 80%
历史数据流失	图片散落各处无法复用，历史项目数据无法追溯或分析	无法做物料质量分析和成本优化

2.2 为什么做成平台而非单一系统

首期需求虽是结案报告自动化，但团队同样面临合同审查、费用报销核对、供应商对账等大量重复性文档处理场景。这些场景共享以下结构：

- 有结构化输入（Excel / PDF / 图片 / 云相册）
- 有明确的校验规则和输出模板
- 流程中有 1～2 个必须人工确认的节点（Human-in-loop）
- 最终产物是一份可下载的报告或文件

若每个场景单独开发，维护成本线性增长。将「Agent 引擎 + Tool 调度 + 状态机 + UI Shell」抽象为平台层，每个业务场景以 Skill 套形式接入，复用率高，新 Skill 接入工期可控在 1～3 周。

2.3 核心目标与量化验收标准

目标维度	目标描述	量化验收标准	测量方式
效率	显著缩短单项目图文处理时间	≤ 30 分钟/项目（含人工标注确认阶段，以 AI 预标注完成为前提）	UAT 实测 3 个真实项目取均值
准确	降低结算漏项风险	结算漏项率 ≤ 10%；系统发出数量警告时拦截率 100%	审计日志统计
凭据	自动化凭据配对核对	自动配对成功率 ≥ 75%；剩余手动处理时间 ≤ 15 分钟/项目	FeedbackEvent 统计
资产	构建结构化项目数字资产	每个结案项目产出完整的物料-图片-坐标-凭据结构化数据集	数据库记录完整性校验
平台	支持新 Skill 快速接入	第二个 Skill 接入工期 ≤ 3 周，无	开发日志验收

目标维度	目标描述	量化验收标准	测量方式
		需修改平台层代码	
性能	关键操作响应流畅	AI 推理单张 ≤ 8s；PPT 生成 ≤ 60s/100 张；缩略图加载 ≤ 2s	压测日志

2.4 核心设计原则

原则	含义	违反的后果
平台层与业务层严格分离	平台代码不包含任何业务逻辑；Skill 代码只通过标准接口与平台交互	Skill 代码散落进平台，导致平台无法独立升级
每个 Skill 完全自包含	Skill A 和 Skill B 互不感知，不共享业务状态，不互相调用 Tool	Skill 间耦合导致一个 Skill 故障拖垮其他 Skill
两级 Agent 隔离	顶层 Agent 只做 Skill 路由；子 Agent 只看见本 Skill 的 Tool 列表	多 Skill 后 Agent Tool 列表膨胀，规划质量下降
LLM 后端完全可插拔	平台通过 LLMAdapter 接口调用模型，具体实现由 .env 配置	模型更换需修改业务代码
Human-in-loop 是一等公民	Agent 在任何节点可暂停/恢复，状态持久化，服务重启后可继续	中断任务导致用户数据丢失，重复操作
安全优先	图片/二进制数据不出内网；文件上传白名单双重校验；所有路径经沙箱验证	数据泄露或 RCE 攻击
可观测性内建	每个任务携带 trace_id，所有 Tool 调用完整记录，关键指标可监控	生产问题无法排查，SLA 无法度量

三、用户角色与权限矩阵

3.1 角色定义

角色	简称	核心职责	典型操作
项目执行人员	Executor	上传素材、完成标注、提交结案	活动结束后整理照片、绑定设计图、做 PPT
结案审核专员	Reviewer	审核 PPT 报告、通过或驳回结案	核查物料数量与现场照片是否对应
财务结算人员	Finance	查看结案报告、核对凭据与结算数据	按报告执行付款或扣款决策
系统管理员	Admin	管理用户/PPT 模板/Skill 配置/LLM 参数/经验层	上传新模板、调整 AI 阈值、查看经验报告

3.2 功能权限矩阵

功能模块	Executor	Reviewer	Finance	Admin
创建项目	✓ 本人	✗	✗	✓ 全部
删除项目	✓ 草稿状态	✗	✗	✓ 全部
上传素材（Excel/图片/云相册）	✓	✗	✗	✓
设计图拖拽绑定	✓	✗	✗	✓
标注画板操作	✓	只读	✗	✓
废片清空（软删除）	✓ 含二次确认	✗	✗	✓
提交审核	✓ 本人项目	✗	✗	✓
审核通过 / 驳回	✗	✓	✗	✓
查看结案 PPT	✓ 本人	✓ 全部	✓ 已完结	✓
下载 PPT	✓ 本人	✓	✓ 已完结	✓
查看凭据配对详情	✓ 本人	✓	✓ 已完结	✓
管理 PPT 模板	✗	✗	✗	✓
配置 AI 阈值	✗	✗	✗	✓
用户管理	✗	✗	✗	✓
经验层管理台	✗	✗	✗	✓
Skill 启用 / 禁用	✗	✗	✗	✓
查看审计日志（脱敏版）	✓ 本人任务	✓	✗	✓ 完整版
查看系统监控指标	✗	✗	✗	✓

行级数据隔离	所有项目查询自动附加 department_id = current_user.department_id 条件，跨部门数据在 ORM QuerySet 层过滤。Admin 可查全部。

四、系统架构总览

4.1 三层平台架构

层次	组成模块	职责	修改权限
Layer 1 平台核心（Platform Core）	Agent 引擎 / LLM Adapter / Tool 调度器 / 任务状态机 / CancellationToken / ContextManager / OutputGuard / StreamParser / 权限系统 / 通知系统 / 审计日志 / Shared Toolkit	所有 Skill 共用的基础设施，提供一致性保障	仅平台团队维护，Skill 开发者不得修改
Layer 2 Skill 注册表（Skill Registry）	skill.json 描述文件 / agent_prompt.md / tools/*.py / ui/组件 / tests/	每套业务 Skill 的完整自包含实现，互相隔离	各 Skill 独立开发，遵守注册协议（见第六章）
Layer 3 Web UI Shell（前端容器）	顶部导航 / Skill 选择器 / Agent 状态浮层 / Human-in-loop 插槽 / 通知中心 / 版本历史 / Admin 后台（含经验层管理台）	统一用户界面容器，按需加载各 Skill UI 组件	前端团队维护容器；Skill 通过 Module Federation 注入组件

4.2 两级 Agent 编排

两级隔离是防止多 Skill 紊乱的核心机制。无论接入多少 Skill，顶层 Agent 永远只有一个 Tool（activate_skill），规划质量不随 Skill 数量增长而下降。

Agent 层级	职责	可见的 Tool	生命周期
顶层 Agent（Orchestrator）	接收用户自然语言输入，判断用户意图，选择目标 Skill，传递初始上下文	仅 activate_skill(skill_id: str, context: dict) 一个 Tool	每次对话创建，选定 Skill 后销毁
Skill 子 Agent（Skill Agent）	在 Skill 上下文内完整规划执行步骤，调用 Tool，处理异常，在 HIL 节点暂停等待人工操作	仅限本 Skill 在 skill.json 中注册的 Tool 列表；平台内置 Tool 自动注入（get_task_status 等）	由顶层 Agent 激活，任务 COMPLETED/CANCELLED/ERROR 时销毁；WAITING_HUMAN 时挂起保留上下文

4.3 Human-in-loop（HIL）节点完整机制

步骤	执行方	技术实现	用户感知
1. 声明 HIL 需求	Skill 子 Agent	调用 request_human_input(ui_component="AnnotationCanvas", prefill_data={...})	Agent 状态浮层从 RUNNING 变 WAITING_HUMAN
2. 任	平台层	任务状态写 Redis（key: task:{id}:status =	顶部出现橙色横幅「需

步骤	执行方	技术实现	用户感知
务挂起持久化		WAITING_HUMAN）；完整 context_json 写 TaskContext 表；TaskCheckpoint 写入最新步骤	要您操作：完成物料标注」
3. HIL 界面弹出	前端 Shell	路由跳转至 /tasks/{id}/annotate；从 prefill_data 中读取 AI 候选框作为初始状态渲染画板	画板打开，AI 候选框已预填，用户可直接确认或修改
4. 用户完成操作	用户	在画板上确认/修改标注，点击「提交标注」	提交后看到 loading 状态，随后返回任务进度页
5. 幂等性保护	平台层	POST /tasks/{id}/resume 先获取 Redis 分布式锁（SETNX，TTL=60s）；成功才恢复；失败返回 409 避免重复唤醒	用户重复点击提交不会触发多次 Agent
6. 任务恢复	平台层	从 TaskCheckpoint 读取最新检查点上下文，注入人工操作结果；任务状态变 RUNNING	Agent 状态浮层恢复为执行中，显示下一步骤
7. 子 Agent 继续	Skill 子 Agent	以人工标注结果为输入继续调用后续 Tool	进度继续推进，直到下一个 HIL 节点或 COMPLETED

HIL 上下文摘要	HIL 节点触发时，系统在操作界面顶部展示 reasoning_summary（≤200 字），说明 Agent 为什么需要人工介入，如「AI 已完成预标注，检测到 15 个低置信候选框需确认，以下是待处理项：」

4.4 任务状态机

状态	进入条件	可转移到	持久化位置	服务重启处理
CREATED	用户发起任务	RUNNING	Redis + MySQL AgentTask	—
RUNNING	Agent 开始执行 / HIL 恢复	WAITING_HUMAN / COMPLETED / ERROR / CANCELLED	Redis（实时）+ MySQL（定期刷写）	标记为 ERROR，提示用户重试
WAITING_HUMAN	Agent 调用 request_human_input	RUNNING（用户提交后）/ CANCELLED	Redis + MySQL + TaskCheckpoint	状态保留，用户返回后可继续操作
COMPLETED	所有 Tool 执行完毕	— 终态	MySQL	—
ERROR	不可恢复异常 / 超过 max_steps / 超过 max_tokens	RUNNING（手动重试）	MySQL + 错误详情日志	—
CANCELLED	用户主动取消	— 终态	MySQL	CancellationToken 传播中断 GPU/LLM 调用

4.5 Agent 状态浮层 UI 规格

Agent 状态	浮层形式	位置	用户可操作项
IDLE	不显示	—	—
RUNNING	右下角悬浮胶囊：旋转圆圈 + 当前步骤名称（如「AI 正在检测物料…」）+ 百分比进度	固定右下角，z-index:200	点击展开步骤日志抽屉；点击「取消」中止任务
WAITING_HUMAN	顶部全宽橙色横幅：「需要您操作：[步骤名]」，包含 reasoning_summary 摘要	固定顶部，z-index:200	点击横幅跳转操作界面；右侧「稍后处理」收起横幅但不取消任务
COMPLETED	绿色 Toast 通知（3s 自动消失）：「[项目名] 结案报告已生成，点击查看」	右上角，z-index:400	点击跳转结果页
ERROR	红色横幅（持续显示）：错误摘要 + 错误码 + 「重试」按钮	固定顶部，z-index:200	重试（重新从最新 Checkpoint 恢复）/ 查看详情 / 复制错误码

五、平台层详细规格

5.1 LLM Adapter 接口规格

平台通过统一抽象接口调用 LLM，后端开发人员通过环境变量切换实现，业务 Skill 代码完全不感知底层模型。

```
# platform/llm/base.py
from abc import ABC, abstractmethod
from typing import List, Optional, AsyncIterator

class Message(BaseModel):
    role: Literal["system","user","assistant","tool"]
    content: List[ContentBlock]  # TextBlock | ImageBlock | ToolResultBlock

class ContentBlock(BaseModel):
    type: Literal["text","image","tool_result"]
    text: Optional[str]
    image_data: Optional[str]  # base64，仅本地模型调用，禁止发往外部 API
    image_url: Optional[str]   # 仅允许内网 URL

class LLMAdapter(ABC):
    @abstractmethod
    async def complete(self, messages: List[Message], tools: List[ToolDef] = None) -> LLMResponse: ...
    @abstractmethod
    async def stream(self, messages: List[Message], tools: List[ToolDef] = None) -> AsyncIterator[LLMChunk]: ...
```

适配器	对应模型	.env 配置	多模态支持	外部 API
ClaudeAdapter	Anthropic Claude（推荐）	LLM_ADAPTER=claude ANTHROPIC_API_KEY=sk-ant-xxx MODEL_ID=claude-sonnet-4-5-20251001	文字 ✓ 图片 ✗（禁止发送）	是
OpenAIAdapter	OpenAI GPT-4o 或兼容接口	LLM_ADAPTER=openai OPENAI_API_KEY=sk-xxx OPENAI_BASE_URL=https://...	文字 ✓ 图片 ✗（禁止发送）	是
OllamaAdapter	本地 Ollama（Qwen2.5/Llama3）	LLM_ADAPTER=ollama OLLAMA_BASE_URL=http://localhost:11434 OLLAMA_MODEL=qwen2.5:7b	文字 ✓ 图片 ✓（本	否

适配器	对应模型	.env 配置	多模态支持	外部 API
			地安全）	
InferenceAdapter	本地 GPU 推理服务（见 5.3）	LLM_ADAPTER=inference INFERENCE_BASE_URL=http://localhost:8001	图片检测专用	否

安全约束	DataClassifier 在 LLMAdapter 调用前检查 payload：若包含 image_data（base64 图片二进制），强制路由到本地 InferenceAdapter，拒绝发送至外部 API。违规调用记录安全日志并告警。

5.2 LLM 工程化组件

5.2.1 OutputGuard（输出格式守卫）

所有 Tool 定义 Pydantic 输出 Schema。LLM 返回结果先经 OutputGuard 校验，失败时发送 Self-Correction Prompt，最多重试 3 次。

```
class OutputGuard:
    async def validate_and_correct(self, raw_output: str, schema: Type[BaseModel],
                                   context: str, max_retries: int = 3) -> BaseModel:
        for attempt in range(max_retries):
            try:
                return schema.model_validate_json(raw_output)
            except ValidationError as e:
                if attempt == max_retries - 1:
                    raise AgentOutputError(f"格式校验失败 {max_retries} 次: {e}")
                raw_output = await self.self_correct(raw_output, e, context)
```

5.2.2 ContextManager（上下文窗口管理）

长链路执行中防止 Token 爆炸：

- 每个 Tool 返回结果存入数据库，只向 Agent 返回 tool_result_id 和摘要（≤500 tokens）
- Tool 输出超过 500 tokens 时，由 ContextManager 调用 LLM 生成结构化摘要
- Agent 基于摘要做决策，原始数据持久化在 TaskCheckpoint；Agent 不处理原始大数据
- 每次 LLM 调用前，ContextManager 计算当前消息总 Token 数；超过阈值（默认 60K tokens）时触发历史摘要压缩

5.2.3 StreamParser（流式输出解析器）

处理 LLM 流式输出中的 Tool 调用参数拼接：

```
class StreamParser:
    # 状态机: IDLE → IN_TOOL_CALL → ACCUMULATING_JSON → COMPLETE
    # 积累 JSON 片段直到出现完整的 } 闭合，再解析 Tool 调用参数
    # 解析失败时回退到完整输出后统一解析（非逐 token 解析）
```

5.2.4 熔断器与降级策略

触发条件	熔断行为	降级策略	恢复机制
外部 API 连续失败 3 次	熔断开关置 OPEN，拒绝后续调用	自动切换到 fallback_adapter（.env 中配置 LLM_FALLBACK=ollama）	30 分钟后置 HALF_OPEN，单次探测成功后置 CLOSED
外部 API 响应超时	单次调用超时（默认 30s）	httpx.AsyncClient.timeout 控制；超时计入失败次数	—
Agent 执行超限	任务步骤数 > max_steps（50）或总 Token > max_tokens（200K）	强制终止任务，状态变 ERROR，清理临时文件	用户手动重试

5.3 本地 AI 推理服务规格

视觉标注（AI 预标注）独立为单一 GPU 服务进程，所有 Celery Worker 通过 HTTP 队列调用，GPU 显存只加载一次。

```
# 推理服务接口规范（OpenAI Compatible 格式）
POST http://localhost:8001/v1/detect
{
  "image_base64": "...",      // 原始图片 base64，内网安全
  "queries": ["对讲机", "主舞台背板"],  // 来自 Excel 的物料文本名称
  "confidence_threshold": 0.4  // 由 SkillConfig 动态配置
}

# 响应
{
  "boxes": [
    {"query": "对讲机", "bbox": {"x":0.12,"y":0.30,"w":0.25,"h":0.18},
     "confidence": 0.85, "after_nms": true}
  ]
}
```

技术规格	说明
模型选型	首选 Grounding DINO（开源，零样本，适合非标物料）；备选 Qwen-VL-Int4（本地推理更快，中文物料名理解更好）；由 .env DETECTION_MODEL 配置
NMS 后处理	推理输出经 IoU 阈值=0.5 的非极大值抑制（NMS）去除冗余重叠框；跨类别去重保留最高置信度框
EXIF 归一化	上传时所有图片用 Pillow.ImageOps.exif_transpose() 物理消除 EXIF 旋转；存储归一化后的版本；坐标基于归一化图计算，消除 iPhone 竖拍旋转导致的坐标系错位
批量处理	Celery Worker 逐张发送 HTTP 请求到推理服务；单张超时 30s 自动跳过标记 is_ai_failed=true；整体进度通过 Redis Pub/Sub 推送前端
显存管理	GPU 推理服务进程唯一，Worker 数量不影响显存占用；服务启动时预加载模型到显存；请求排队由推理服务内部队列控制

5.4 Tool 调度器规格

功能	实现方式
Tool 注册	Skill 加载时调用 ToolDispatcher.register(skill_id, tool_list)，Tool 注册到隔离命名空间 {skill_id}::{tool_name}
路由调用	dispatch(skill_id, tool_name, input, context)：校验 tool_name 归属当前 skill_id；跨 Skill 调用返回 PermissionError 并记录安全日志
超时控制	每个 Tool 在 skill.json 中配置独立 timeout（默认 30s）；超时返回标准错误对象 {success:false, error_type:"TIMEOUT"}，不抛出异常
重试策略	网络类错误（httpx.NetworkError）自动重试 3 次，指数退避+随机抖动；业务逻辑错误（error_type=BUSINESS）不重试，直接返回 Agent 决策
错误分类	ToolResult.error_type 分三类：BUSINESS（业务逻辑失败，Agent 处理）/ SYSTEM（系统级崩溃，触发告警）/ TIMEOUT（超时，Agent 决定是否重试）
调用审计	每次调用写 ToolCallLog：tool_name / input 摘要（前 500 字符）/ output 摘要 / duration_ms / error_type / trace_id；敏感字段（金额/发票号）脱敏
幂等键	任务级别幂等：idempotency_key = SHA256(user_id + file_hashes + skill_id)；AgentTask 表唯一索引；重复提交返回已有任务 ID

5.5 平台共享工具包（Shared Toolkit）

为避免各 Skill 重复实现相同基础能力，平台层提供以下开箱即用工具：

模块路径	提供函数	说明
platform.shared.ocr	extract_text_from_image(img_path) → str extract_structured_fields(img_path, field_schema) → dict	混合管线：pdfplumber 提取文字 → 正则匹配 → 失败降级 LLM Vision；两路交叉验证；不一致字段标 low_confidence
platform.shared.pdf	extract_text_from_pdf(pdf_path) → str extract_tables_from_pdf(pdf_path) → List[DataFrame]	pdfplumber 文字 + camelot 表格；扫描版 PDF 自动调用 OCR 管线
platform.shared.excel	parse_spreadsheet(file_path, column_hints) → DataFrame detect_column_mapping(df, hint_list) → dict	支持 .xlsx(openpyxl) / .xls(xlrd) / .csv(pandas) / .ods(odf)；合并单元格自动展开；列名模糊匹配有歧义时返回候选列供 Agent 提问
platform.shared.pptx	generate_pptx(template_id, slides_data) → str add_annotation_shapes(slide, annotations) → None	读取 PPTTemplate 表配置（不依赖备注区）；坐标转换；标注形状渲染；中文字体 Noto Sans CJK SC
platform.shared.vision	classify_image(img_path, categories) → dict detect_objects(img_path, text_queries) → List[Box]	调用本地推理服务；图片不出内网
platform.shared.storage	save_file(project_id, file, type) → FileRecord get_signed_url(file_id, expires_in) → str	内容寻址（SHA256 去重）；统一路径管理；所有路径经 PathValidator 沙箱校验

模块路径	提供函数	说明
platform.shared.notify	send_notification(user_id, msg, link, channel) → None	站内消息 + 可选企业微信 Webhook；事务提交后异步发送，失败不回滚业务

5.6 安全基线规格

安全域	具体规格
文件上传	白名单：[jpg,jpeg,png,heic,webp,pdf,xlsx,xls,csv,zip]；双重校验（扩展名 + python-magic 文件头 Magic Bytes）；ZIP 解压限制 2GB/1000 文件；图片像素限制 50MP（50000×10000）；拒绝非白名单文件，记录安全日志
路径安全	PathValidator.validate(path, base_dir) 用 pathlib.Path.resolve() 解析后校验是否在允许目录内；所有文件路径禁止直接使用 Agent/用户输入字符串
数据出境	DataClassifier 在发送至外部 API 前检查 payload；包含 image_data（图片二进制）的请求强制路由本地模型；OCR 提取的纯文本字段允许发往外部 API
Prompt 注入防护	InputSanitizer 过滤 OCR/用户输入中的已知注入模式；外部内容作为 data 字段传入 Tool 而非 user message（防止被 LLM 当作指令执行）
IDOR 防护	HIL 恢复接口 POST /tasks/{id}/resume 强制校验 task.creator_id == current_user.id 或 role==admin；不匹配返回 403
审计日志	ToolCallLog 敏感字段（金额/发票号/商户名）脱敏（保留首尾各 2 位，中间替换 *）；完整数据写安全日志文件（按日轮转保留 30 天）；Admin 权限访问
JWT 管理	Access Token 有效期 8h；Refresh Token 30d；前端在过期前 5 分钟自动刷新；过期后拦截请求队列，弹出重新登录对话框，登录后自动重放失败请求
密钥管理	生产环境密钥通过系统环境变量或 Docker Secrets 注入；.env 文件不提交代码库；.env.example 只含 key 名称

5.7 可观测性规格

维度	实现方案	关键指标
结构化日志	structlog + JSON 格式；每条日志携带 trace_id / task_id / skill_id / user_id / tool_name	—
链路追踪	每个 AgentTask 生成全局 trace_id（UUID v4）；贯穿 FastAPI → Celery → Agent → Tool 调用链；随 API 响应头 X-Trace-ID 返回前端展示	—
进度推送	批量 Tool（AI 推理 / OCR / PPT 生成）通过 Redis Pub/Sub 推送细粒度进度 {current, total, step_desc}；前端 WebSocket 订阅，浮层显示百分比进度条	—
TaskWatchdog	Celery Beat 定时任务（每 5 分钟）；扫描 status=RUNNING 且 updated_at < now()-30min 的僵尸任务；发送心跳探测；无响应则标记 ERROR 并触发告警	僵尸任务数
Prometheus 指标	任务成功率 / Tool 调用耗时 P50/P95 / Token 消耗	SLA 保障

维度	实现方案	关键指标
	量 / 磁盘使用率 / GPU 显存使用率	
磁盘配额	每个项目临时文件目录软限制 1GB；ResourceGC（Celery Beat 每小时）清理已完结/取消超 24h 的临时裁切图和缩略图	磁盘安全

六、Skill 注册协议与扩展规范

6.1 Skill 目录结构

```
skills/
  skill-event-report/              ← Skill A（首期：结案报告）
    skill.json                     ← Skill 描述与注册信息（必须）
    agent_prompt.md                ← 子 Agent 系统 Prompt（必须）
    tools/                         ← Tool 实现（必须）
      parse_excel.py
      classify_assets.py
      fetch_cloud_album.py
      run_ai_detection.py
      bind_design_images.py        ← HIL Tool
      request_annotation.py        ← HIL Tool
      validate_quantity.py
      ocr_receipt.py
      match_receipts.py
      request_receipt_confirm.py   ← HIL Tool
      generate_ppt.py
      submit_review.py
    ui/                            ← React 组件（HIL 界面）
      DesignBinder.tsx
      AnnotationCanvas.tsx
      ReceiptMatcher.tsx
    tests/
      test_tools.py
      test_integration.py

  skill-contract-review/           ← Skill B（示例：合同审查）
    skill.json
    ...
```

6.2 skill.json 完整规范

```
{
  "id": "skill-event-report",
  "name": "活动结案报告",
  "version": "1.2.0",
  "min_platform_version": "1.0.0",
  "min_context_version": "1",
  "description": "自动化生成活动物料核对结案 PPT 及凭据报告",
  "icon": "FileBarChart",
  "trigger_examples": ["帮我做结案报告","处理这批活动照片"],
  "required_roles": ["executor"],
  "tools": [
    {"name":"parse_excel",       "timeout":30,  "type":"auto"},
    {"name":"classify_assets",   "timeout":120, "type":"auto"},
    {"name":"fetch_cloud_album", "timeout":300, "type":"auto"},
    {"name":"run_ai_detection",  "timeout":600, "type":"auto","async":true},
    {"name":"bind_design_images","timeout":0,   "type":"human_in_loop","ui":"DesignBinder"},
    {"name":"request_annotation","timeout":0,
"type":"human_in_loop","ui":"AnnotationCanvas"},
    {"name":"validate_quantity", "timeout":10,  "type":"auto"},
    {"name":"ocr_receipt",       "timeout":60,  "type":"auto"},
    {"name":"match_receipts",    "timeout":30,  "type":"auto"},
```

```
{"name":"request_receipt_confirm","timeout":0,"type":"human_in_loop","ui":"ReceiptMatcher"},
    {"name":"generate_ppt",      "timeout":180, "type":"auto"},
    {"name":"submit_review",     "timeout":10,  "type":"auto"}
  ]
}
```

6.3 Tool 函数规范

每个 Tool 是一个 Python 函数，必须遵守以下规范：

```
from platform.tool_base import ToolResult, TaskContext

async def my_tool(input: dict, context: TaskContext) -> ToolResult:
    """
    Args:
        input: Tool 输入参数（由 Agent 生成的 JSON）
        context: 平台注入的上下文（task_id, user_id, logger, storage, llm）
    Returns:
        ToolResult(success, data, error_type, error_code, message, summary)
        summary: ≤200 字的摘要，是 Agent 实际看到的内容（data 存数据库）
    """
    try:
        # ... 实现逻辑
        return ToolResult(success=True, data=result, summary="处理完成：共 N 条记录")
    except BusinessError as e:
        return ToolResult(success=False, error_type="BUSINESS", message=str(e))
    except Exception as e:
        context.logger.error("system_error", exc=str(e), trace_id=context.trace_id)
        return ToolResult(success=False, error_type="SYSTEM", message="内部错误，已记录")
```

规范项	要求
幂等性	同样输入多次调用结果一致；状态写入前查询是否已存在
所有文件操作	强制使用 with open() as f: 上下文管理器，防止句柄泄露
金额计算	所有货币金额使用 from decimal import Decimal，禁止 float 运算
路径操作	禁止直接使用 Agent 输出的字符串作为路径，必须经 PathValidator.validate()
数据库批量写入	≥10 条记录使用 SQLAlchemy bulk_insert_mappings()，单次批量 ≤500 条
错误分类	系统级异常（Exception）返回 SYSTEM 类型触发告警；业务逻辑失败返回 BUSINESS 类型 Agent 处理
日志	使用 context.logger，携带 trace_id；禁止 print()

6.4 新增 Skill 六步流程

1. 在 skills/ 目录创建新目录，编写 skill.json（见 6.2 规范）
2. 编写 agent_prompt.md：说明 Skill 的目标、Tool 使用顺序规则、HIL 节点处理方式、异常处理指引
3. 在 tools/ 下按规范实现每个 Tool 函数（见 6.3 规范）
4. 如有 HIL 节点，在 ui/ 下实现对应 React 组件，遵守 Module Federation 规范（CSS Modules，禁止全局样式污染）

5. 编写 tests/：每个 Tool 至少覆盖正常路径、边界条件、异常路径三类测试
6. 在 Admin 后台「Skill 管理」页启用该 Skill（Feature Flag 控制灰度上线）

热加载	平台启动时自动扫描所有 skill.json 完成注册。新增 Skill 通过 POST /admin/skills/{id}/reload 热加载，无需重启平台服务。热加载使用 Python importlib.import_module 动态导入，失败时旧版本保持可用。

七、首期 Skill A：活动结案报告

7.1 业务流程总览

用户输入「帮我做 [项目名] 的结案报告」或在导航栏选择「活动结案报告」Skill 后，顶层 Agent 激活本 Skill 子 Agent。子 Agent 自动推进全流程，在三个 HIL 节点暂停等待用户操作，其余步骤完全自动执行。

步骤	Tool 名称	执行方式	核心输入	核心输出	HIL?
0	（初始化）	自动	项目信息	项目 ID，AssetsUpload 界面就绪	否
1	parse_excel	自动	报价单文件	物料列表 [{name,category,target_qty,sort_order}]	否（列名歧义时 Agent 对话提问）
2	classify_assets	自动	已上传图片列表	分类结果 {design_render/field_photo/receipt}，低置信标「待确认」	否
3	fetch_cloud_album（可选）	自动	云相册 URL	下载并入库的图片 ID 列表	否
4	bind_design_images	HIL	物料列表 + 设计图列表	物料-设计图映射关系	是（DesignBinder UI）
5	run_ai_detection	异步自动	照片列表 + 物料名列表	候选框列表（NMS 后处理，含 confidence）	否（Celery 队列，进度实时推送）
6	request_annotation	HIL	照片列表 + AI 候选框	确认后的标注坐标列表（相对百分比）	是（AnnotationCanvas UI）
7	validate_quantity	自动	标注数量 + 目标数量	校验报告（通过/警告/拦截）	否（拦截时 Agent 对话提问是否强行继续）
8	ocr_receipt	自动	支付截图 + 发票	结构化凭据 [{type,amount,date,merchant,invoice_no}]	否（低置信字段标记）

步骤	Tool 名称	执行方式	核心输入	核心输出	HIL?
			PDF		
9	match_receipts	自动	支付凭据列表 + 发票列表	初步配对结果 + 未匹配项	否
10	request_receipt_confirm	HIL	配对结果	用户确认后的最终配对表	是（ReceiptMatcher UI）
11	generate_ppt	自动	标注 + 设计图映射 + 凭据配对 + 模板 ID	结案 PPT 文件路径 + 生成摘要	否
12	submit_review（可选）	自动	项目 ID + PPT 路径	审核任务已创建，通知已发送	否

7.2 资产输入规格

资产类型	支持格式	数量限制	说明
报价单	xlsx / xls / csv	1 份	自动解析物料名称、规格、数量、单价列；列名模糊匹配
设计效果图	jpg/jpeg/png/webp/pdf（PPT 需用户手动转换为图片）	≤200 张，单张≤50MB	绑定到对应物料，在 PPT 左侧展示
现场照片（本地上传）	jpg/jpeg/png/heic/webp	≤500 张，单张≤50MB	tus 分片上传（1MB/片），断点续传；上传后异步 EXIF 归一化+缩略图生成
现场照片（云相册）	云相册 URL（支持 Alltuu 等）	最多拉取 1000 张	Playwright 无头浏览器抓取；域名白名单；并发 10 线程下载
支付凭据	jpg/jpeg/png/heic（微信/支付宝截图）	≤100 张	OCR 提取金额、日期、收款方
发票	pdf / jpg/jpeg/png	≤100 份	pdfplumber 提取文字+正则；失败降级 LLM Vision；负数/冲红票支持

7.3 Tool 详细规格

Tool 1：parse_excel — Excel 解析

- 路由策略：.xlsx/.xlsm → openpyxl；.xls → xlrd；.csv → pandas.read_csv（sep 自动检测）；.ods → pandas(odf)
- 合并单元格处理：遍历 merged_cells 属性，将主单元格值复制填充到所有合并单元格，避免读取返回 None
- 列名模糊匹配：计算编辑距离，相似度阈值 0.7；存在多列相似度>0.8 时返回歧义候选列，Agent 提问用户选择
- 排序字段：生成 sort_order（Excel 行号，保持原始顺序）用于 PPT 物料排版顺序
- 输出格式：[{id, name, category, target_qty, unit_price, sort_order}]

Tool 2：classify_assets — 资产自动分类

- 触发时机：图片上传入库后自动执行，不阻塞上传流程
- 分类目标：design_render（设计效果图）/ field_photo（现场照片）/ receipt（支付凭据/发票）
- 实现：调用本地 InferenceAdapter LLM Vision 做三分类，置信度低（<0.6）时在步骤 4 DesignBinder UI 中高亮待确认，用户可一键修正分类
- 结果写入：AssetImage.asset_type 字段

Tool 3：fetch_cloud_album — 云相册抓取

- 输入：url（相册链接）、password（可选）、filter_keywords（可选，如过滤视频缩略图）
- 技术实现：Playwright headless Chromium 加载页面，防反爬；提取所有 img src 中的原始高清 URL（排除缩略图 URL pattern）
- 翻页：自动翻页直到无下一页按钮，最多抓取 1000 张
- 下载：asyncio 并发 10 线程下载到项目临时目录，限速 50MB/s；单张失败跳过记录日志
- 安全：URL 域名经白名单校验（Admin 可配置允许域名）；下载文件进入标准上传流程（文件头校验 + EXIF 归一化）
- 进度：CloudAlbumFetch 表记录抓取任务状态，WebSocket 推送「已下载 X/Y 张」

Tool 4：bind_design_images — 设计图绑定（HIL）

- UI 组件：DesignBinder
- 左侧：物料树状列表（含目标数量、classify_assets 的未确认分类标记）
- 右侧：未绑定设计图缩略图瀑布流（虚拟滚动，react-virtual），支持多选
- 操作：拖拽或点击绑定右侧图片至左侧物料节点；已绑定显示绿色角标；支持解绑
- 验证：提交前检查未绑定物料，弹黄色警告（可跳过，PPT 对应位置留设计图占位符）
- 结果写入：Item.design_image_path

Tool 5：run_ai_detection — AI 视觉预标注

- 架构：Celery 异步批量任务；逐张发送 HTTP 请求到本地 InferenceServer；Worker 间不竞争 GPU 显存
- 输入：image_id 列表 + item_names 列表（来自 Excel 物料名，中文文本直接作为 text query）
- NMS 后处理：IoU 阈值 0.5 去重叠框；同位置跨类别保留最高置信度框
- 输出分级：confidence ≥ 0.7 → 绿色框默认勾选；0.4~0.7 → 黄色框默认不选；< 0.4 → 丢弃

- 单张超时：30s 自动跳过，AssetImage.is_ai_failed = true；整体进度 Redis Pub/Sub → WebSocket → 前端进度条
- Tool 返回摘要：「已处理 450/500 张，高置信候选框 312 个，低置信 89 个，未覆盖 99 张」

Tool 6：request_annotation — 标注画板（HIL）

UI 组件：AnnotationCanvas（Fabric.js 5.x）

- 左侧进度树：物料列表 + 实时计数（已标注/目标），点击筛选右侧图片；绿色=达标，橙色=不足，红色=未开始
- 右侧图片区：虚拟滚动（IntersectionObserver 驱动），同时渲染 ≤20 张；单张点击进入标注模式
- 工具栏（Fabric.js Canvas）：
  - 矩形框（默认）：绘制后可拖拽/缩放；右键菜单绑定到不同物料条目
  - 红线工具：起止两点拉直线，可附加文字标签（如「灯带约 100m」）
  - 圆圈工具：绘制椭圆，专门圈选发票凭证关键数字
  - 局部放大：标注框面积 < 总图 5% 时系统自动弹出「生成放大图」建议（用户可拒绝）；放大图位置在原图右下角，用虚线指引线连接
  - 橡皮擦：删除选中标注（不影响其他标注）
  - 撤销/重做：Command Pattern 差分快照（20 步），不做全量快照避免内存泄露
- AI 候选框展示：绿色（高置信）/黄色（低置信）预填；可点击确认或拖拽调整；确认后标记 is_confirmed=true，source 保持 ai
- 坐标存储：强制相对百分比 {x,y,w,h}；line 类型存 {x1,y1,x2,y2}；circle 存 {cx,cy,r}；geometry_json BLOB 存原始几何，x/y/w/h 冗余存外接矩形供快速查询
- 自动暂存：标注状态每 30s 写入 IndexedDB；路由切换时检查未提交草稿，提示「有未提交标注，是否恢复？」
- 坐标空间保真：缩略图统一 Contain 缩放模式，AssetImage 存 thumb_scale/thumb_offset_x/thumb_offset_y；画板坐标转换时用三个参数映射，确保与原图对齐

Tool 7：validate_quantity — 数量校验

- 对比每个 Item 的标注数量（is_confirmed=true 的 Annotation 数）与 target_qty
- 误差 ≤ 10%：直接通过，继续下一步
- 误差 > 10%：返回 BUSINESS 错误，Agent 通过对话提问用户：「[对讲机] 数量不足(8/12)，财务可能拒审，是否继续生成？」；用户确认后方可继续
- 结果写入 QuantityCheckResult，PPT 生成时在对应物料页添加数量标注

Tool 8：ocr_receipt — 凭据 OCR 识别

- 输入路由：PDF → pdfplumber 提取文字 + 正则匹配金额/日期/发票号（精确）；图片截图 → LLM Vision 识别
- 双路交叉验证：若 PDF 文字提取和 LLM Vision 结果不一致，置信度标记为 low_confidence
- 支持负数/冲红发票：Receipt.amount 允许 DECIMAL(12,2) 负值；冲红票在 UI 中红色标识
- 提取字段：amount（含税，Decimal）/ date（datetime）/ merchant_name / invoice_no / receipt_type（payment/invoice）
- 异步批量：Celery 并行处理；进度 WebSocket 推送

Tool 9：match_receipts — 凭据自动配对

- 一阶段：精确金额匹配（amount 完全相等）+ 日期差 ≤ 60 天 → 高置信配对，绿色
- 二阶段：子集和匹配（多张发票金额之和 ≈ 支付金额，允许 Decimal("5.00") 误差）；参与票数限制 ≤ 5 张防 NP-Hard 爆炸
- 冲红抵消：先将同商户正负票净额计算，净额参与配对
- 未匹配项标红，进入步骤 10 的 ReceiptMatcher UI 手动处理

Tool 10：request_receipt_confirm — 凭据配对确认（HIL）

- UI 组件：ReceiptMatcher
- 布局：左栏支付截图缩略图列表，右栏发票缩略图列表，中间 SVG 连线（绿色=自动配对，灰色=手动）
- 操作：拖拽支付截图到发票建立连线；点击连线删除；未匹配项红色高亮
- 底部提示：「X 项支付未配对，Y 张发票未配对」；所有项处理完毕才允许提交
- 支持「标记为无对应发票」操作（如差旅杂项支出），减少强制配对压力

Tool 11：generate_ppt — PPT 生成

模板管理

- PPTTemplate 表存储模板元数据（不依赖备注区配置）：name / file_path / page_capacity / layout_type / font_name / is_active
- Admin 上传模板时执行 TemplateValidator：遍历占位符尝试写入测试文本，失败则提示 Admin 在 PowerPoint 中解除母版保护
- Executor 生成时从下拉菜单选择模板，默认使用上次选择

生成规格

- 物料排版：按 Item.sort_order 顺序；每物料一组幻灯片：左侧设计效果图 + 右侧实物图拼图 + 标注叠加层
- 坐标裁切：在原图（非缩略图）上执行；向外扩张 5～10% Padding；边缘越界 clamp：max(0, x-pad) / min(1-x+pad, w+2*pad)
- 标注叠加层渲染到 PPT（python-pptx）：
  - 矩形框 → add_shape(MSO_SHAPE.RECTANGLE, 红色边框 2pt，无填充)
  - 指引线 → add_connector(MSO_CONNECTOR.STRAIGHT) + 箭头 + TextBox
  - 圆圈 → add_shape(MSO_SHAPE.OVAL, 红色边框)
  - 局部放大图 → 裁切局部区域 → 缩放到宽 300px → 放置在原图右下角 + 虚线指引
- 分页算法：实物图数量 > page_capacity 时克隆版式页；克隆页标题填「[物料名]（续 N/M）」；克隆不复制封面/页眉元素
- PPT 文本溢出处理：物料名超出占位符时先尝试缩小字号（最小 8pt）；仍溢出则截断+「...」；记录警告日志
- 中文字体：强制使用 Noto Sans CJK SC（docker-compose 安装 fonts-noto-cjk）
- 凭据汇总页：按 ReceiptMatch 表生成支付截图+发票缩略图并排展示；标注金额/日期/配对类型
- 占位图：无配图物料强制填充「待补齐实物图」占位图，不中断生成
- 文件原子写入：生成到临时文件（fcntl.flock 文件锁），完成后 os.rename 原子替换
- 版本快照：生成后写入 ProjectSnapshot 表；保留最近 5 版本；超出自动清理旧文件

Tool 12：submit_review — 提交审核

- 更新 Project.status = pending_review，记录 ReviewLog（action=submit）
- 事务 COMMIT 后异步发送站内通知给 Reviewer（事务失败不回滚通知，通知失败不回滚业务）
- 通知内容：项目名称 + 提交人 + 生成 PPT 文件大小 + 快速审核链接

7.4 审核流转状态机

状态	描述	可转移到	触发角色	通知对象
draft	草稿，尚未提交审核	pending_review	Executor	—
pending_review	已提交，待审核	approved / rejected	Reviewer	Reviewer（提交时）
rejected	审核驳回，需修改	pending_review（重提）	Reviewer	Executor（驳回时）
approved	审核通过，结案完成	— 终态	Reviewer	Executor + Finance（通过时）

- 驳回必填原因（≥10 字）；驳回后 Executor 收到通知：项目名 + 驳回原因摘要 + 重新提交链接
- 审核历史：每次操作写 ReviewLog；Finance 可查看完整审核历史
- 多次驳回：Project.reject_count 原子递增（SQL: reject_count = reject_count + 1）

7.5 业务规则完整清单

规则 ID	规则名称	规则详述
R-01	数量校验容错	误差≤10%直接通过；>10% Agent 对话提问，用户确认才继续；AI 候选框未经人工确认不计入数量校验
R-02	凭据日期容差	支付日期与发票日期差≤60 天为有效配对；超出标「日期异常」需用户确认
R-03	凭据金额精度	所有金额使用 DECIMAL(12,2)；子集和容差 Decimal("5.00")；浮点比较禁止直接相等
R-04	冲红发票处理	负数发票（冲红）先与同商户正数票抵消，净额参与配对；ReceiptMatcher UI 中红色标识
R-05	废片软删除	清空无标注照片为软删除（is_deleted=true），二次确认弹窗显示待清空数量；回收站保留 7 天可恢复，7 天后 ResourceGC 物理清理
R-06	PPT 可编辑格式	生成物强制为 .pptx；业务接受代码排版的毫米级误差，用户可下载后手动微调
R-07	文件内容寻址	上传时计算 SHA256 哈希；相同内容不重复存储；路径含哈希前缀防缓存混淆
R-08	坐标边界保护	裁切扩张 Padding 时 clamp 到 [0,1]，不报错不越界

八、经验层（Experience Layer）

8.1 设计理念

经验层是平台的元学习基础设施。核心使命：通过积累用户操作数据，持续改进 Agent Prompt 和 Tool 参数，并自动发现来自开源社区的新能力。经验层不是独立 AI 模型，而是「反馈采集 → 模式挖掘 → 规格建议 → 人工确认」的闭环机制，确保系统在使用中越来越准确。

8.2 五子模块架构

子模块	职责	触发时机	输出物
FeedbackCollector（反馈采集器）	捕获所有 HIL 节点的用户操作与 AI 预测的差异	每次用户在 HIL 界面完成操作后	FeedbackEvent 记录（写入数据库）
PatternMiner（模式挖掘器）	分析 FeedbackEvent，找出 Agent 高频出错的场景和规律	FeedbackEvent 累积 ≥50 条（可配置）	PatternReport：错误类型统计 + 典型反例摘要
PromptTuner（Prompt 调优器）	基于 PatternReport 生成 Prompt/ 参数改进建议，供 Admin 审核采纳	PatternReport 生成后自动触发	Prompt diff 建议 + 预期改进说明
ToolDiscovery（工具发现器）	监控 GitHub/HuggingFace，发现与本 Skill 互补的开源工具	每周定时任务	候选 Tool 列表 + 集成评估报告
BranchManager（分支管理器）	管理针对特定场景的 Agent 配置变体	Admin 手动创建或 PromptTuner 建议	AgentBranch 配置记录

8.3 FeedbackCollector — 采集的事件类型

事件类型	触发时机	记录内容
annotation_added	用户在 AI 未检测的位置新增了标注框	image_id / item_id / 坐标 / 图片区域特征（颜色直方图摘要）
annotation_corrected	用户移动或缩放了 AI 候选框	original_bbox / corrected_bbox / original_confidence
annotation_rejected	用户取消勾选了 AI 高置信候选框	被拒绝的 bbox / confidence / 用户可选填的原因
receipt_matched_manually	用户手动建立了支付-发票配对	payment_amount / invoice_amount / date_diff_days
excel_column_remapped	用户修正了 Excel 列名自动映射	original_guess / correct_column / file_sample_header
quantity_override	用户在数量警告后选择强行继续	item_name / expected_qty / actual_qty / override_reason（可选）
classification_corrected	用户修正了 classify_assets 的分类结果	original_type / corrected_type / image_id

8.4 PromptTuner — 建议机制

PromptTuner 不自动修改 Prompt，只生成「建议」供 Admin 审核：

- 读取 PatternReport 中的高频错误场景
- 定位到 agent_prompt.md 中对应的 Tool 描述段落
- 调用 LLM 生成 diff 格式的修改建议（+ 新增内容，- 删除内容）
- Admin 在「经验层 → Prompt 建议」页面查看：接受 / 修改后接受 / 拒绝（填写原因）
- 所有决策写入 PromptTuningHistory 表，形成审计链

8.5 ToolDiscovery — 开源工具发现

搜索源	检索方式	过滤条件
GitHub API	按 topic 关键词搜索（Admin 配置，如 invoice-ocr / event-photo / receipt-parser）	Stars ≥ 100；最近 6 个月有提交；License 为 MIT/Apache/BSD
HuggingFace Hub	按任务类型搜索（document-question-answering / object-detection）	月下载量 ≥ 1000；有推理 API
Papers with Code	按关键词搜索最新 SOTA 方法	有对应开源代码

发现结果在 Admin 后台「工具发现」页面展示：仓库链接 / 能力描述 / 与现有 Tool 的关系（互补/替代）/ 集成难度（Easy/Medium/Hard）。Admin 标记「感兴趣」后，系统生成基础集成代码骨架（Tool 函数模板 + skill.json 条目）。

8.6 AgentBranch — 场景化 Agent 分支

针对特定业务场景（如大型演唱会 vs 小型展会）维护差异化配置：

```
AgentBranch {
  id, skill_id, name,            // 如「大型演唱会专用」
  base_prompt_version,           // 基准 Prompt 版本
  prompt_overrides: JSON,        // 只记录与 base 的 diff
  tool_param_overrides: JSON,    // 如 {ai_confidence_threshold: 0.3}
  usage_count, last_used_at      // 使用统计
}
```

Executor 触发任务时可选择分支；默认使用 default 分支；分支列表在任务创建页展示。Admin 可基于 FeedbackCollector 数据评估各分支效果。

九、Harness 工程结构与 CI/CD

9.1 适配性评估

Harness 能力	适用性	建议
CI Pipeline（持续集成）	强烈推荐	自动化 lint + test + docker build；PR 触发；测试覆盖率门控 ≥ 70%
Feature Flags（功能开关）	强烈推荐	控制新 Skill 灰度上线；秒级回滚；按 department_id/user_id 白名单开放
CD Pipeline（持续交付）	推荐	SSH 部署到内网服务器；docker-compose 滚动重启；无需 Blue/Green（单节点）
Chaos Engineering	V2 Backlog	系统稳定后引入故障注入测试；当前优先功能完整性
Cloud Cost Management	可选	如外部 LLM 费用成为关注点可接入；当前规模手动统计即可

9.2 Feature Flags 与 Skill 上线流程

阶段	操作	Harness 行为
开发完成	代码合并到 main 分支	CI Pipeline 自动运行：lint → test → build → push registry
内测	创建 Feature Flag「skill_{id}」，仅 Admin 可见	前端 Skill 选择器读取 Flag 状态，非白名单用户不显示该 Skill
扩大测试	Flag 白名单扩展到指定 department_id	Executor 可在任务创建页看到并使用新 Skill
全量上线	Flag 全量开启	所有角色可见新 Skill
紧急回滚	一键关闭 Flag	用户立即看不到该 Skill，无需重新部署，≤5s 生效

9.3 CI/CD Pipeline 规格

```yaml
# harness-pipeline.yaml（内网 Harness Community Edition）
pipeline:
  name: AI-Copilot-Platform
  stages:
    - stage:
        name: Test
        steps:
          - run: pip install -r requirements.txt --quiet
          - run: black --check platform/ skills/
          - run: flake8 platform/ skills/ --max-line-length=120 --extend-ignore=E203
          - run: pytest tests/ --cov=platform --cov-fail-under=70 -x -q
    - stage:
        name: Build
        when: branch == "main"
        steps:
          - buildAndPushDockerRegistry:
              connectorRef: internal-registry
```

```
            imageName: aicopilot-platform
            tags: ["${HARNESS_BUILD_ID}", "latest"]
  - stage:
      name: Deploy
      when: branch == "main"
      steps:
        - run: |
            ssh deploy@${INTERNAL_SERVER} \
              "cd /opt/aicopilot && \
               docker-compose pull && \
               docker-compose up -d --no-deps --build api worker && \
               docker-compose exec api alembic upgrade head"
```

轻量替代	若不使用 Harness：CI 用 Gitea Actions（内网 Gitea）或 GitHub Actions；Feature Flags 用 SkillConfig 表 is_enabled 字段（Admin 后台切换）；CD 用 Watchtower 监听镜像更新自动重启。

十、数据库完整设计

10.1 平台层核心表

表名	核心字段	说明
User	id UUID PK / name / email UNIQUE / password_hash（bcrypt）/ role enum(executor/reviewer/finance/admin) / department_id / is_active / created_at / updated_at	平台用户；department_id 用于行级数据隔离
AgentTask	id / skill_id / user_id FK / idempotency_key UNIQUE / status enum(CREATED/RUNNING/WAITING_HUMAN/COMPLETED/ERROR/CANCELLED) / current_step / max_steps(50) / total_tokens_used / trace_id / created_at / updated_at	每次 Agent 执行记录；status 对应第四章状态机
TaskContext	id / task_id FK UNIQUE / context_json BLOB / schema_version / updated_at	从 AgentTask 分离，避免主表 JSON 列频繁更新页分裂
TaskCheckpoint	id / task_id FK / step_index / step_name / tool_name / output_summary / next_step / schema_version / created_at	Agent 执行检查点；HIL 恢复从最新 Checkpoint 读取上下文，不读全量
ToolCallLog	id / task_id FK / tool_name / input_digest（SHA256）/ input_summary（前 500 字）/ output_summary（前 500 字）/ duration_ms / error_type / status / trace_id / created_at	审计日志；完整 input/output 写文件日志（按日轮转）；数据库只存摘要
Notification	id / user_id FK / task_id FK / type / content / link / is_read / created_at	站内消息；所有 Skill 共用
SkillConfig	id / skill_id / config_key / config_value / updated_by FK / updated_at	Skill 运行时参数（如 AI 阈值、Feature Flag is_enabled）

10.2 Skill A 业务表

表名	核心字段	关键设计
Project	id / project_no UNIQUE / name / client_id FK / creator_id FK / reviewer_id FK / department_id / status enum / reject_count INT DEFAULT 0 / pptx_path / created_at / updated_at	reject_count 用 SQL 原子递增；status 对应 7.4 审核状态机
Item	id / project_id FK / name / category / target_qty	sort_order 来自 Excel 行

表名	核心字段	关键设计
	INT / unit_price DECIMAL(12,2) / design_image_path / sort_order INT / created_at	号；决定 PPT 物料排版顺序
AssetImage	id / project_id FK / asset_type enum(design_render/field_photo/receipt/unknown) / original_path / thumbnail_path / thumb_scale / thumb_offset_x / thumb_offset_y / file_size / mime_type / sha256_hash UNIQUE / exif_normalized BOOL / is_deleted BOOL DEFAULT false / deleted_at / source enum(upload/cloud_album) / upload_time	sha256_hash 实现内容寻址；thumb_scale/offset 用于坐标系对齐；is_deleted 软删除
Annotation	id / image_id FK / item_id FK / type enum(rect/line/circle) / x,y,w,h FLOAT4（外接矩形，相对百分比）/ geometry_json BLOB（原始几何）/ content VARCHAR(200) / source enum(ai/human) / confidence FLOAT / is_confirmed BOOL DEFAULT false / created_by FK / created_at	geometry_json 存原始几何（{x1,y1,x2,y2} / {cx,cy,r}）；x,y,w,h 冗余外接矩形供查询
Receipt	id / project_id FK / asset_image_id FK / receipt_type enum(payment/invoice) / amount DECIMAL(12,2) / txn_date DATE / merchant_name / invoice_no / ocr_confidence / low_confidence_fields JSON / is_void BOOL / created_at	amount 支持负值（冲红）；low_confidence_fields 记录识别不确定的字段
ReceiptMatch	id / project_id FK / payment_id FK / invoice_id FK / match_type enum(auto_exact/auto_subset/manual/unmatched) / amount_diff DECIMAL(12,2) / date_diff_days INT / confirmed_by FK / created_at	amount_diff 用 DECIMAL；支持 unmatched 标记（无对应发票）
ReviewLog	id / project_id FK / operator_id FK / action enum(submit/approve/reject) / comment VARCHAR(1000) / created_at	驳回必填 comment（≥10 字）；不可删除，为财务审计证据
PPTTemplate	id / name / file_path / page_capacity INT / layout_type / font_name / tags JSON / is_active BOOL / uploaded_by FK / created_at	替代备注区配置；Admin 上传时执行 TemplateValidator 预检
ProjectSnapshot	id / project_id FK / version INT / pptx_path / file_size / generated_at / triggered_by_step	PPT 版本历史；保留最近 5 版；超出自动清理旧文件
CloudAlbumFetch	id / project_id FK / source_url / platform / total_count / downloaded_count / failed_count / status / started_at / completed_at	云相册抓取任务记录
QuantityCheckResult	id / project_id FK / item_id FK / target_qty / actual_qty / delta_pct / status enum(pass/warn/block) / override_by FK / override_reason / checked_at	数量校验结果；override 记录用户确认记录

10.3 经验层表

表名	核心字段	说明

表名	核心字段	说明
FeedbackEvent	id / skill_id / task_id FK / event_type enum / before_state JSON / after_state JSON / user_comment / created_at	HIL 节点用户操作差异采集
PatternReport	id / skill_id / analysis_type / summary / sample_count / suggestions_json / created_at	模式挖掘结果报告
PromptTuningHistory	id / skill_id / suggestion_summary / action enum(accepted/modified/rejected) / applied_diff TEXT / reason / decided_by FK / created_at	Prompt 调优决策审计链
AgentBranch	id / skill_id / name / base_prompt_version / prompt_overrides JSON / tool_param_overrides JSON / usage_count / last_used_at / created_by FK	Agent 场景化分支配置

10.4 数据库运维规范

规范项	具体要求
迁移工具	引入 Alembic；每次 Schema 变更生成版本化迁移脚本；docker-compose up 时自动执行 alembic upgrade head
时区统一	所有 DateTime 字段存储 UTC；MySQL timezone=+00:00；容器 TZ=UTC；前端展示时转换用户本地时区
并发安全	统计类字段（reject_count）用 SQL 原子操作（UPDATE ... SET x = x + 1）；禁止先读后写
索引策略	AgentTask(user_id, status, created_at)；ToolCallLog(task_id, created_at)；AssetImage(project_id, is_deleted)；Annotation(image_id, item_id)；Receipt(project_id, receipt_type)
Celery Worker 连接池	在 worker_process_init 信号中调用 db.engine.dispose() 重新初始化连接池，防止 fork 导致连接混乱
批量写入	≥10 条 Annotation/Receipt 使用 bulk_insert_mappings()；单次批量 ≤500 条；避免逐条 INSERT 耗尽连接池

十一、部署架构与环境配置

11.1 服务组件清单

组件	技术选型	职责	部署位置
Web 前端	React 18 + Vite + Module Federation	UI Shell + Skill UI 组件，浏览器直接访问	Nginx 静态托管 / CDN（内网）
API 服务	Python 3.11 + FastAPI + uvicorn	平台 API + Skill Tool 端点	内网 GPU 服务器
Agent 执行引擎	Python asyncio + LLMAdapter	两级 Agent 编排逻辑	与 API 服务同进程（V1）
异步任务队列	Celery 5.x + Redis 7.x	图片处理 / AI 推理 / OCR 异步任务	内网服务器 + Redis 独立容器
本地 AI 推理服务	Grounding DINO / Qwen-VL + FastAPI（独立进程）	视觉预标注，唯一 GPU 进程，显存只加载一次	GPU 服务器独立进程（port 8001）
数据库	MySQL 8.x + Alembic	业务数据持久化	内网服务器
文件存储	本地文件系统（/{project_uuid}/{sha256[:8]}/）	原图/缩略图/PPT（V1 单节点；V2 可替换 MinIO）	NAS 或本地 SSD
网页抓取	Playwright Chromium（无头）	云相册图片抓取	API 服务器（独立 Chrome 进程）
外部 LLM API（可选）	Anthropic / OpenAI（仅文字类任务）	Agent 规划 + 文字 OCR（图片不出内网）	外网，内网代理访问

11.2 docker-compose.yml 关键配置要点

```
services:
  api:
    build: .
    init: true          # 使用 tini 处理 SIGTERM，防止僵尸进程
    ulimits:
      nofile: {soft: 65536, hard: 65536}  # 防文件描述符耗尽
    environment:
      - LLM_ADAPTER=claude
      - ANTHROPIC_API_KEY=${ANTHROPIC_API_KEY}
      - DATABASE_URL=mysql+aiomysql://...
      - REDIS_URL=redis://redis:6379/0
      - FILE_STORAGE_ROOT=/data/aicopilot
      - TZ=UTC
    command: >
      sh -c "alembic upgrade head &&
             uvicorn main:app --host 0.0.0.0 --port 8000"

  worker:
```

```
    build: .
    command: celery -A worker worker --loglevel=info --concurrency=4
    environment: [*api_env]

  inference:
    build: ./inference_server
    deploy:
      resources:
        reservations:
          devices: [{driver: nvidia, count: 1, capabilities: [gpu]}]
    ports: ["8001:8001"]

  nginx:
    image: nginx:alpine
    volumes:
      - ./frontend/dist:/usr/share/nginx/html
      - ./nginx.conf:/etc/nginx/nginx.conf
    # nginx.conf 中设置 proxy_read_timeout 3600s（防长耗时 Agent 响应超时）
```

11.3 混合部署策略

任务类型	推荐 LLM	理由
Agent 规划 / 推理	外部 Claude API（推荐）	规划能力强；多步推理准确；按 Token 计费；单项目约 ¥15～25 元（月 50 项目约 ¥750～1250 元）
视觉标注（AI 预标注）	本地 Grounding DINO	500 张图计算量大；现场照片含敏感内容不宜外传；本地 RTX 4090 每张 ≤ 8s
凭据 OCR（图片类）	本地 InferenceAdapter LLM Vision	凭据含财务金额属敏感数据，不出内网
凭据 OCR（PDF 纯文字）	规则引擎（pdfplumber + 正则）优先	纯结构化数据无需 LLM；速度更快更准
资产分类（classify_assets）	本地 InferenceAdapter	图片不出内网；三分类任务本地模型足够

11.4 .env 配置完整示例

```
# ── LLM 配置（后端开发人员填写）──────────────────
LLM_ADAPTER=claude
ANTHROPIC_API_KEY=sk-ant-xxxxx
MODEL_ID=claude-sonnet-4-5-20251001
LLM_FALLBACK=ollama          # 熔断时降级到本地 Ollama
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:7b

# ── 本地 AI 推理 ──────────────────────────────
DETECTION_MODEL=grounding-dino
INFERENCE_BASE_URL=http://localhost:8001
AI_CONFIDENCE_HIGH=0.7
AI_CONFIDENCE_LOW=0.4

# ── 数据库 ────────────────────────────────────
```

```
DATABASE_URL=mysql+aiomysql://aicopilot:password@mysql:3306/aicopilot
REDIS_URL=redis://redis:6379/0

# ── 存储 ──────────────────────────────────────
FILE_STORAGE_ROOT=/data/aicopilot
MAX_UPLOAD_SIZE_MB=50
PROJECT_DISK_QUOTA_GB=1

# ── Agent 限制 ────────────────────────────────
AGENT_MAX_STEPS=50
AGENT_MAX_TOKENS=200000

# ── 安全 ──────────────────────────────────────
SECRET_KEY=<随机生成，不提交代码库>
ACCESS_TOKEN_EXPIRE_HOURS=8
REFRESH_TOKEN_EXPIRE_DAYS=30
```

十二、非功能性需求与 UAT 验收

12.1 性能指标

指标	要求	测试场景
缩略图加载	800px WebP ≤ 2s	500 张照片，内网 100Mbps
AI 单张推理	≤ 8s（RTX 4090，Grounding DINO）	图片分辨率 ≤ 12MP
凭据 OCR	≤ 5s/张（PDF 纯文字 ≤ 1s）	含 LLM Vision 识别路径
PPT 生成	100 张实物图 ≤ 60s	含裁切 + 标注叠加 + 凭据页全流程
Agent 规划首步	用户输入后 ≤ 3s 开始第一步工具调用	顶层路由 + 子 Agent 启动
并发任务	≥ 5 个项目同时运行 AI 推理	Celery Worker ≥ 5 并发，推理服务内部排队
WebSocket 心跳	客户端每 30s 发 PING，服务端 30s 内回 PONG；90s 无响应自动重连	长耗时 HIL 等待场景

12.2 可用性与稳定性

规范项	要求
系统可用性	内网工作日 SLA ≥ 99%（每月最多 7 小时计划维护）
数据持久性	项目数据（原图/标注/PPT）保留至项目完结后 ≥ 2 年；废片回收站 7 天
单点故障隔离	AI 推理服务宕机不影响标注画板和 PPT 生成；外部 LLM API 故障降级到本地 Ollama
WAITING_HUMAN 持久化	服务重启后 WAITING_HUMAN 任务状态保留，用户返回后可继续操作（从 TaskCheckpoint 恢复）
RUNNING 重启处理	RUNNING 状态在服务重启后标记为 ERROR；TaskWatchdog 5 分钟后检测并提示用户重试

12.3 浏览器兼容性

环境	支持级别
Chrome 110+（PC）	完全支持，主力浏览器
Edge 110+（PC）	完全支持
Safari 16+（iPad）	文件上传功能支持（用于现场回来后即刻上传）；标注画板功能待 V3 专项支持触屏
移动端浏览器（手机）	仅支持查看报告和通知；标注操作不支持
IE / 旧版 Edge	不支持

12.4 UAT 验收场景

编号	场景描述	预期结果	验收前提
UAT-01	20 种物料+300 张本地照片+10 张凭据，完整走完结案流程	端到端 ≤ 30 分钟（含 HIL 操作）；PPT 含所有物料页+凭据页	AI 预标注（步骤 5）需提前在上传后触发
UAT-02	标注数量缺口 > 10%，点击提交到数量校验步骤	Agent 对话提问而非静默继续；用户拒绝后不生成 PPT；用户确认后继续	—
UAT-03	上传模糊手写金额发票图片	OCR 对应字段标 low_confidence；ReceiptMatcher 中高亮显示待确认	—
UAT-04	云相册 URL 导入（Alltuu 格式），完整抓取并入库	CloudAlbumFetch 记录 100% 下载成功；图片出现在标注画板	准备测试相册 URL
UAT-05	按第六章 6.4 步骤接入任意 Skill B	不修改平台代码；热加载后导航出现新 Skill；可正常创建任务	已开发 Skill B 测试包
UAT-06	切换 LLM_ADAPTER=ollama 后运行结案流程	功能正常；可能规划质量略低但不报错；凭据 OCR 仍走本地路径	已安装本地 Ollama + qwen2.5:7b
UAT-07	服务重启时有 WAITING_HUMAN 任务	重启后 Agent 状态浮层恢复橙色横幅；用户进入操作界面可继续完成	—
UAT-08	Finance 角色尝试访问标注画板	返回 403；导航中无标注入口	—
UAT-09	清空废片后进入回收站恢复一张照片	照片重新出现在画板图片列表；is_deleted 变 false	—
UAT-10	上传包含 .exe 文件的 ZIP 包	系统拒绝上传并返回错误提示；安全日志记录一条告警	—
UAT-11	PPT 重新生成后查看版本历史	ProjectSnapshot 表新增一条；历史版本列表显示 2 个版本；可下载旧版本	已有第一次生成的版本
UAT-12	经验层：标注修改后触发 PatternReport	FeedbackEvent 达到阈值后 PatternReport 自动生成；Admin 后台可查看	需预先累积 ≥50 条 FeedbackEvent

十三、AI 辅助开发实操指南

13.1 总体原则

本项目适合由 1～2 名开发人员主导，全程使用 Claude Code CLI 辅助开发。核心分工：AI 负责生成代码框架和模板代码；开发人员负责架构决策、接口定义和验收测试。

推荐工具组合	Claude Code CLI（claude 命令）用于编码实现；Claude.ai 对话用于架构讨论、PRD 澄清和复杂设计推导；两者分工明确，效率最高。

13.2 开发阶段与 Claude Code 提示词指南

阶段 0：项目骨架初始化（0.5 天）

- 目标：docker-compose up 可启动，所有接口返回 mock 数据

```
claude "请帮我初始化一个 Python FastAPI 项目，目录结构：
platform/（llm/, tool_dispatcher.py, state_machine.py, context_manager.py,
          output_guard.py, stream_parser.py, shared/）
skills/（skill_loader.py）
frontend/（React 18 + Vite，包含空 UI Shell 骨架）
inference_server/（独立 FastAPI，mock /v1/detect 接口）
包含 docker-compose.yml（FastAPI + MySQL + Redis + Celery + Nginx）
包含 alembic 初始化和空 migrations/ 目录
所有接口先返回 mock 数据，确保 docker-compose up 后能访问前端首页和 API 健康检查"
```

阶段 1：平台层核心（3～4 天）

- LLM Adapter（含多模态支持）：

```
claude "实现 LLMAdapter 抽象基类，Message 支持 List[ContentBlock]（TextBlock/ImageBlock）。
实现 ClaudeAdapter 和 OllamaAdapter。
实现 DataClassifier：调用前检查 payload 是否含 image_data；
包含则强制路由本地模型，拒绝发送到外部 API，记录安全日志。
从 .env LLM_ADAPTER 自动选择实现。包含完整 pytest-mock 单元测试。"
```

- OutputGuard + ContextManager：

```
claude "实现 OutputGuard，用 Pydantic 校验 LLM 输出，失败时发送 Self-Correction Prompt，
最多重试 3 次，3 次失败抛出 AgentOutputError。
实现 ContextManager，Tool 输出超 500 tokens 时摘要，向 Agent 只返回摘要和 tool_result_id。
写完整单元测试。"
```

- 熔断器：

```
claude "实现 CircuitBreaker 类，连续失败 3 次置 OPEN，30 分钟后 HALF_OPEN 探测。
状态存 Redis（key: circuit_breaker:{adapter_name}）。
OPEN 时调用 fallback_adapter。写单元测试覆盖 OPEN/HALF_OPEN/CLOSED 转换。"
```

- Tool 调度器 + 任务状态机：

```
claude "实现 ToolDispatcher：命名空间隔离、路由校验、超时控制（asyncio.wait_for）、
错误分类（BUSINESS/SYSTEM/TIMEOUT）、ToolCallLog 写入（input/output 摘要+SHA256）。
实现 TaskStateMachine：状态持久化 Redis+MySQL、pause_for_human（Redis SETNX 幂等锁）、
resume（Checkpoint 恢复）、CancellationToken 传播。写集成测试。"
```

阶段 2：Skill A 所有 Tool（5～7 天，按 Tool 逐一开发）

- parse_excel Tool（含合并单元格处理）：

```
claude "实现 parse_excel Tool，支持 xlsx/xls/csv/ods 四种格式路由。
合并单元格展开：遍历 ws.merged_cells 将主单元格值填充到所有合并格。
列名模糊匹配（编辑距离，阈值 0.7），歧义时返回 needs_clarification=true 和候选列。
输出 [{id,name,category,target_qty,unit_price,sort_order}]。写完整 pytest 测试。"
```

- run_ai_detection Tool（含 NMS）：

```
claude "实现 run_ai_detection Tool，Celery 异步批量处理。
每张图调用 InferenceServer POST /v1/detect（HTTP，超时 30s，失败标 is_ai_failed）。
NMS 后处理（IoU=0.5）+ 跨类别去重。进度 Redis Pub/Sub 推送。
返回摘要，原始候选框存数据库返回 result_id。写 mock HTTP 服务的集成测试。"
```

- ocr_receipt Tool（混合管线）：

```
claude "实现 ocr_receipt Tool，混合管线：
PDF 路径：pdfplumber 提取文字 → 正则匹配金额/日期/发票号 → 失败降级 LLM Vision。
图片路径：直接调用本地 InferenceAdapter LLM Vision（图片不出内网）。
两路结果交叉验证，不一致字段标 low_confidence。
amount 用 Decimal。负数/冲红票支持。写完整测试含 PDF 和图片两路。"
```

- generate_ppt Tool（坐标映射 + 标注渲染）：

```
claude "实现 generate_ppt Tool，从 PPTTemplate 表读取配置（不用备注区）。
坐标转换：shape_left = photo_left + ann.x * photo_width（见第七章公式）。
标注渲染：rect→add_shape RECTANGLE 红色边框；line→add_connector+TextBox；circle→add_shape OVAL。
局部放大图：bbox 面积<5%时生成，裁切+缩放 300px+右下角放置+虚线指引。
分页：页数 = ceil(图片数/page_capacity)，克隆版式页标题加（续 N/M）。
中文字体 Noto Sans CJK SC；文本溢出缩至最小 8pt 或截断+...
文件原子写入（flock + os.rename）；版本快照写 ProjectSnapshot。写集成测试。"
```

阶段 3：前端 UI Shell + Skill UI（5～6 天）

- Module Federation 配置（防组件冲突）：

```
claude "配置 Webpack 5 Module Federation：平台 Shell 为 Host，Skill UI 为 Remote。
每个 Skill 在独立的 Vite 项目中开发，打包为 Remote 模块。
所有 Skill CSS 使用 CSS Modules，class 名加 skill-{id}__ 前缀，禁止全局样式污染。
Shell 的 lazy import 示例代码。"
```

- Agent 状态浮层 + WebSocket（含心跳）：

```
claude "实现 AgentStatusBar React 组件。
WebSocket 订阅 /ws/task/{id}，30s PING/PONG 心跳，90s 超时自动重连，重连后 HTTP 同步状态（Hydration）。
RUNNING：右下角胶囊+进度百分比，点击展开步骤日志 Drawer。
WAITING_HUMAN：顶部橙色横幅+reasoning_summary，点击跳转 HIL 路由。
JWT Token 在过期前 5 分钟自动刷新，过期后拦截请求队列弹登录框。"
```

- 标注画板（最复杂，建议独立对话）：

```
claude "实现 AnnotationCanvas React + Fabric.js 5.x 组件。
虚拟滚动（react-virtual）：同时渲染 ≤20 张图，IntersectionObserver 懒加载。
```

```
Undo/Redo 用 Command Pattern 差分快照（20 步），禁止全量快照。
坐标转换：从缩略图坐标空间到相对百分比，使用 thumb_scale/offset 三参数映射。
自动暂存：30s 写 IndexedDB，路由切换前检查草稿。
局部放大建议：bbox 面积<5%弹建议 Toast，用户可接受/拒绝。
tus 大文件上传集成（1MB/片，断点续传）。"
```

阶段 4：集成联调（3～4 天）

- 编写 Skill A agent_prompt.md（这一步自己写初稿，再用 AI 优化）：

```
claude "我有以下 agent_prompt.md 草稿：[粘贴内容]
请优化：确保 12 个 Tool 调用顺序清晰；明确标注 HIL 节点（bind_design_images/
request_annotation/request_receipt_confirm）处 Agent 需暂停并说明原因。
添加异常处理指引：parse_excel 列名歧义时如何提问用户；
validate_quantity 数量不足时如何提问；ocr_receipt 置信度低时如何说明。
确保摘要描述清晰（Agent 看摘要不看原始大 JSON）。"
```

- 端到端集成测试：

```
claude "写 pytest 集成测试，mock：LLM API（返回预设 Tool 调用序列）、
InferenceServer（返回 mock 候选框）、文件系统（tmpdir）、外部 HTTP。
测试完整流程：parse_excel → classify_assets → run_ai_detection →
[HIL mock: 提交预设标注] → validate_quantity → ocr_receipt →
match_receipts → [HIL mock: 确认配对] → generate_ppt → submit_review。
验证：每个状态转换正确；TaskCheckpoint 写入完整；ToolCallLog 记录每次调用；PPT 文件已生成。"
```

阶段 5：安全加固与验收（2 天）

- 安全自检：

```
claude "请 review 以下上传接口代码 [粘贴代码]，检查：
1. 文件白名单是否同时校验扩展名和 Magic Bytes（python-magic）
2. ZIP 解压是否有大小限制（2GB）和文件数限制（1000）
3. 所有文件路径是否经过 PathValidator（pathlib resolve + base_dir 检查）
4. 是否有 DataClassifier 防止图片发往外部 LLM API"
```

13.3 Claude Code 使用技巧

场景	推荐方式
开始新 Tool 开发	先在 claude.ai 描述清楚输入/输出/边界条件，确认方案后再在 Claude Code 中生成；避免直接说「实现 X 功能」
调试复杂报错	贴完整错误栈 + 相关代码 +「期望行为 vs 实际行为」；不要只贴错误信息
Fabric.js 画板	独立开一个 Claude Code 对话，不要在超长对话里反复修改（上下文过多会降低代码质量）
写测试	先让 Claude Code 列出所有边界场景（正常/异常/边界），确认后再生成测试代码
架构决策	在 claude.ai 对话中讨论（如「状态机用 Redis 还是只用 MySQL」），不要让 Claude Code 自行决定架构
代码评审	每完成一个模块，用 Claude Code review：「找出潜在的并发安全问题、内存泄露、未处理异常路径」

场景	推荐方式
Celery 相关	Celery 与 SQLAlchemy、Redis 的组合有很多隐坑（连接池/序列化/超时），遇到问题先搜索具体错误信息再问 Claude Code

13.4 里程碑与工期

里程碑	内容	参考工期	验收标准
M0	骨架初始化	0.5 天	docker-compose up 启动；前端首页可访问；API 健康检查返回 200
M1	平台层核心（LLM Adapter + 熔断 + Tool 调度 + 状态机 + 检查点）	3～4 天	所有单元测试通过；mock LLM 可完成多 Tool 调用链；WAITING_HUMAN 暂停/恢复正常
M2	Skill A 所有 Tool 单独测试通过	5～7 天	每个 Tool 的 pytest 测试全绿（含边界和异常路径）
M3	前端 UI Shell + AnnotationCanvas + ReceiptMatcher + Agent 状态浮层	5～6 天	HIL 界面可正常操作；坐标输出格式正确；Module Federation 加载无冲突
M4	端到端集成联调	3～4 天	mock 数据跑通完整 12 步流程；状态机转换全部正确；ToolCallLog 完整
M5	安全加固 + 真实数据 UAT 验收 + 上线	2～3 天	12 个 UAT 场景全部通过；3 个真实项目验收，均 ≤30 分钟完成

参考总工期：19～25 天（1 名全栈开发 + Claude Code CLI 辅助）；2 名并行可缩至 12～15 天。

十四、未来 Skill 扩展路线图

14.1 平台二期优化（V2，Skill A 稳定后）

方向	具体内容
AI 推理增强	将已绑定的设计图作为 Few-shot Visual Reference 传入 Grounding DINO/Qwen-VL，提升非标物料召回率
Agent 路由升级	Skill 数量 > 15 时，基于 sentence-transformers 向量检索替换 Prompt 硬编码路由
Prompt 管理台	PromptTuner 建议可视化界面；支持版本对比、A/B 测试（基于 AgentBranch）；Prompt 变更记录完整追溯
进程隔离	Agent 引擎独立为单独进程（python -m agent_engine），通过 Redis Queue 与 API 服务通信，解决 GIL 竞争

14.2 未来 Skill 套路线图

Skill 名称	业务场景	平台 Shared Toolkit 复用	预估接入工期
Skill B：合同审查	上传甲乙方合同，AI 标注风险条款，生成审查报告	shared.pdf.extract_text / shared.notify	2～3 周
Skill C：费用报销核对	上传报销单 Excel + 凭据，自动核对并生成审批摘要	shared.excel.parse / shared.ocr（大量复用 Skill A 凭据逻辑）	1～2 周
Skill D：供应商对账	导入采购订单与发票，自动配对差异并生成对账表	shared.excel.parse / shared.ocr	1～2 周
Skill E：活动效果分析	基于结案数据生成活动 ROI 分析报告	shared.storage / shared.pptx	2～3 周
Skill F：物料库管理	跨项目物料图片和坐标数字资产复用查询	平台 AgentTask 历史数据	3～4 周

14.3 开放问题与待决策事项

编号	问题	影响范围	建议方向	决策期限
Q-01	PPT 初期支持几套模板？是否强制只用一套？	PPTTemplate 表 + Admin UI	建议 V1 支持 1～3 套，Admin 管理	需求评审前
Q-02	通知渠道是否接入企业微信/钉钉？	通知系统	站内消息优先，企微集成列为 V2	需求评审前
Q-03	Grounding DINO vs Qwen-VL 最终选型	AI 推理模块	需 AI 工程师在 RTX 4090 上做基准测试后决定	V1 开发前
Q-04	HEIC 图片（iPhone 拍摄）是否纳入 V1 支持	文件上传	建议纳入，用 pillow-heif 转 JPEG 后处理	需求评审前

编号	问题	影响范围	建议方向	决策期限
Q-05	云相册域名白名单初始配置哪些平台	fetch_cloud_album Tool	初始支持 alltuu.cc，Admin 可扩展	V1 开发前

— 文档结束 —

AI-Copilot Platform PRD V3.0 | 完整融合版 | 如有修订请更新版本号并注明变更内容